Software Architecture & Design Introduction
Software Architecture

Architecture serves as a **blueprint for a system**. It provides an abstraction to manage the system complexity and establish a communication and coordination mechanism among components.

- It defines a **structured solution** to meet all the technical and operational requirements, while optimizing the common quality attributes like performance and security.
- Further, it involves a set of significant decisions about the organization related to software development and each of these decisions can have a considerable impact on quality, maintainability, performance, and the overall success of the final product. These decisions comprise of −
    - Selection of structural elements and their interfaces by which the system is composed.
    - Behavior as specified in collaborations among those elements.
    - Composition of these structural and behavioral elements into large subsystem.
    - Architectural decisions align with business objectives.
    - Architectural styles guide the organization.

Software Design

Software design provides a **design plan** that describes the elements of a system, how they fit, and work together to fulfill the requirement of the system. The objectives of having a design plan are as follows −

- To negotiate system requirements, and to set expectations with customers, marketing, and management personnel.
- Act as a blueprint during the development process.

Difference between Software Design and Software Architecture
1. Software Design:

Software design refers to the process of creating a specification of software artifact which will help to developers to implement the software. It is about designing individual modules/components means it defines what is module is performing, the classes, functions and their usages, etc. It is considered one of the initial phase of Software Development Life Cycle (SDLC). Software design process establishes a best possible design plan to implement the system. It mainly focuses more on internal components of the system and their interaction with each other.

2. Software Architecture:
Software architecture refers to fundamental structure or the process of creating high level structure of a software system. Initially software architecture is created and when it is agreed then that software design is carried out. In software architecture the solution is structured in such a way so that it meets technical and operational requirements and during structuring application quality, maintenance, performance and overall success all these things are kept in mind as ultimately software architecture impact all these things at last.

S.No.	SOFTWARE DESIGN	SOFTWARE ARCHITECTURE
01.	Software design is about designing individual modules/components.	Software architecture is about the complete architecture of the overall system.
02.	Software design defines the detailed properties.	Software architecture defines the fundamental properties.
03.	In general it refers to the process of creating a specification of software artifact which will help to developers to implement the software.	In general it refers to the process of creating high level structure of a software system.
04.	It helps to implement the software.	It helps to define the high level infrastructure of the software.
06.	Software design is more about on individual module/component.	Software architecture is more about the design of entire system.
07.	It is considered as one initial phase of Software Development Cycle (SSDLC) and it gives detailed idea to developers to implement consistent software.	It is a plan which constrains software design to avoid known mistakes and it achieves one organizations business and technology strategy.
09.	In one word the level of software design is implementation.	In one word the level of software architecture is structure.
10.	How we are building is software design.	What we are building is software architecture.

Goals of Architecture

The primary goal of the architecture is to identify requirements that affect the structure of the application. A well-laid architecture reduces the business risks associated with building a technical solution and builds a bridge between business and technical requirements.

Some of the other goals are as follows −

- Expose the structure of the system, but hide its implementation details.
- Realize all the use-cases and scenarios.
- Try to address the requirements of various stakeholders.
- Handle both functional and quality requirements.
- Improve quality and functionality offered by the system.

- Improve external confidence in either the organization or system.

Limitations of Software Architecture

Software architecture is still an emerging discipline within software engineering. It has the following limitations −

- Lack of tools and standardized ways to represent architecture.
- Lack of analysis methods to predict whether architecture will result in an implementation that meets the requirements.
- Lack of awareness of the importance of architectural design to software development.
- Lack of understanding of the role of software architect and poor communication among stakeholders.
- Lack of understanding of the design process, design experience and evaluation of design.

Role of Software Architect

A Software Architect provides a solution that the technical team can create and design for the entire application. A software architect should have expertise in the following areas −

Design Expertise

- Expert in software design, including diverse methods and approaches such as object-oriented design, event-driven design, etc.
- Lead the development team and coordinate the development efforts for the integrity of the design.
- Should be able to review design proposals and tradeoff among themselves.

Domain Expertise

- Expert on the system being developed and plan for software evolution.
- Assist in the requirement investigation process, assuring completeness and consistency.
- Coordinate the definition of domain model for the system being developed.

Technology Expertise

- Expert on available technologies that helps in the implementation of the system.
- Coordinate the selection of programming language, framework, platforms, databases, etc.

Methodological Expertise

- Expert on software development methodologies that may be adopted during SDLC (Software Development Life Cycle).
- Choose the appropriate approaches for development that helps the entire team.

Hidden Role of Software Architect

- Facilitates the technical work among team members and reinforcing the trust relationship in the team.
- Information specialist who shares knowledge and has vast experience.
- Protect the team members from external forces that would distract them and bring less value to the project.

Deliverables of the Architect

- A clear, complete, consistent, and achievable set of functional goals
- A functional description of the system, with at least two layers of decomposition
- A concept for the system
- A design in the form of the system, with at least two layers of decomposition

- A document or process which ensures functional decomposition is followed, and the form of interfaces is controlled

Note : Refer Class lectures for All Open Ended Discussions

Week 2

UML

UML stands for Unified Modeling Language. It is a pictorial language used to make software blueprints. UML was created by Object Management Group (OMG). The UML 1.0 specification draft was proposed to the OMG in January 1997. It serves as a standard for software requirement analysis and design documents which are the basis for developing a software.

UML can be described as a general purpose visual modeling language to visualize, specify, construct, and document a software system. Although UML is generally used to model software system, it is not limited within this boundary. It is also used to model non software systems such as process flows in a manufacturing unit.

The elements are like components which can be associated in different ways to make a complete UML picture, which is known as a **diagram**. So, it is very important to understand the different diagrams to implement the knowledge in real-life systems. We have two broad categories of diagrams and they are further divided into sub-categories i.e. **Structural Diagrams** and **Behavioral Diagrams**.

Structural Diagrams

Structural diagrams represent the static aspects of a system. These static aspects represent those parts of a diagram which forms the main structure and is therefore stable.

These static parts are represented by classes, interfaces, objects, components and nodes. Structural diagrams can be sub-divided as follows −

- Class diagram
- Object diagram
- Component diagram
- Deployment diagram
- Package diagram

The following table provides a brief description of these diagrams −

Sr.No.	Diagram & Description
1	**Class**

	Represents the object orientation of a system. Shows how classes are statically related.
2	**Object** Represents a set of objects and their relationships at runtime and also represent the static view of the system.
3	**Component** Describes all the components, their interrelationship, interactions and interface of the system.
4	**Package** Describes the package structure and organization. Covers classes in the package and packages within another package.
5	**Deployment** Deployment diagrams are a set of nodes and their relationships. These nodes are physical entities where the components are deployed.

Behavioral Diagrams

Behavioral diagrams basically capture the dynamic aspect of a system. Dynamic aspects are basically the changing/moving parts of a system. UML has the following types of behavioral diagrams −

- Use case diagram
- Sequence diagram
- Communication diagram
- State chart diagram
- Activity diagram
- Interaction overview
- Time sequence diagram

The following table provides a brief description of these diagram −

Sr.No.	Diagram & Description
1	**Use case**

Describes the relationships among the functionalities and their internal/external controllers. These controllers are known as actors.

2 **Activity**

Describes the flow of control in a system. It consists of activities and links. The flow can be sequential, concurrent, or branched.

3 **State Machine/state chart**

Represents the event driven state change of a system. It basically describes the state change of a class, interface, etc. Used to visualize the reaction of a system by internal/external factors.

4 **Sequence**

Visualizes the sequence of calls in a system to perform a specific functionality.

5 **Interaction Overview**

Combines activity and sequence diagrams to provide a control flow overview of system and business process.

6 **Communication**

Same as sequence diagram, except that it focuses on the object's role. Each communication is associated with a sequence order, number plus the past messages.

Architecture View Model

A model is a complete, basic, and simplified description of software architecture which is composed of multiple views from a particular perspective or viewpoint.

A view is a representation of an entire system from the perspective of a related set of concerns. It is used to describe the system from the viewpoint of different stakeholders such as end-users, developers, project managers, and testers.

4+1 View Model

The 4+1 View Model was designed by Philippe Kruchten to describe the architecture of a software–intensive system based on the use of multiple and concurrent views. It is a **multiple view** model that addresses different features and concerns of the system. It standardizes the software design documents and makes the design easy to understand by all stakeholders.

It is an architecture verification method for studying and documenting software architecture design and covers all the aspects of software architecture for all stakeholders. It provides four essential views −

- **The logical view or conceptual view −** It describes the object model of the design.
- **The process view −** It describes the activities of the system, captures the concurrency and synchronization aspects of the design.
- **The physical view −** It describes the mapping of software onto hardware and reflects its distributed aspect.
- **The development view −** It describes the static organization or structure of the software in its development of environment.

This view model can be extended by adding one more view called **scenario view** or **use case view** for end-users or customers of software systems. It is coherent with other four views and are utilized to illustrate the architecture serving as "plus one" view, (4+1) view model. The following figure describes the software architecture using five concurrent views (4+1) model.

Why is it called 4+1 instead of 5?

The **use case view** has a special significance as it details the high level requirement of a system while other views details — how those requirements are realized. When all other four views are completed, it's effectively redundant. However, all other views would not be possible without it. The following image and table shows the 4+1 view in detail −

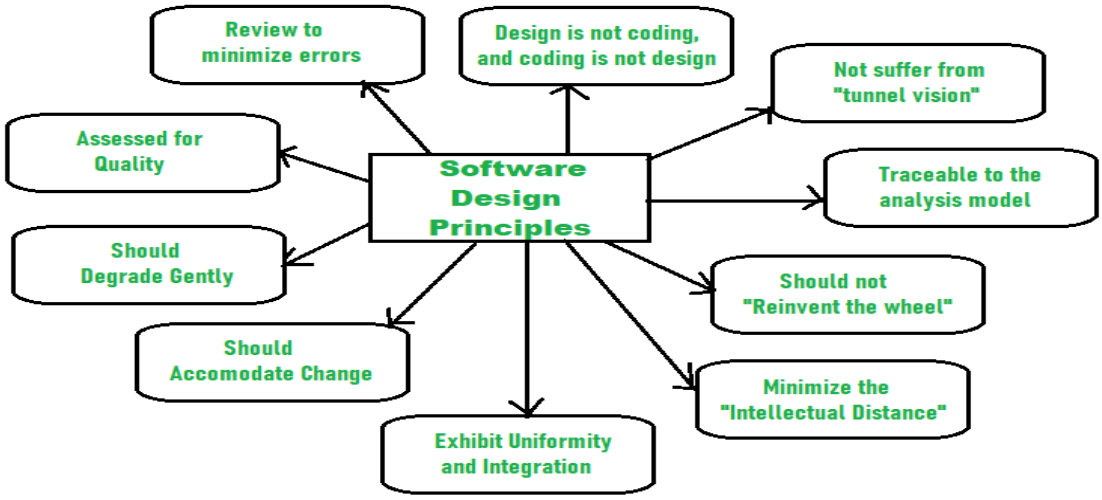
	Logical	**Process**	**Development**	**Physical**	**Scenario**
Description	Shows the component (Object) of system as well as their interaction	Shows the processes / Workflow rules of system and how those processes communicate, focuses on dynamic view of system	Gives building block views of system and describe static organization of the system modules	Shows the installation, configuration and deployment of software application	Shows the design is complete by performing validation and illustration
Viewer / Stake holder	?	?	?	?	?

Consider	Functional requirements	Non Functional Requirements	Software Module organization (Software management reuse, constraint of tools)	Nonfunctional requirement regarding to underlying hardware	System Consistency and validity
UML – Diagram	?	?	?	?	Use case diagram

Note : Refer Class lectures for All Open Ended Discussions

WEEK 3

Design means to draw or plan something to show the look, functions and working of it. **Software Design** is also a process to plan or convert the software requirements into a step that are needed to be carried out to develop a software system. There are several principles that are used to organize and arrange the structural components of Software design. Software Designs in which these principles are applied affect the content and the working process of the software from the beginning. These principles are stated below :



Principles Of Software Design :

1. **Should not suffer from "Tunnel Vision"** – While designing the process, it should not suffer from "tunnel vision" which means that is should not only focus on completing or achieving the aim but on other effects also.

2. **Traceable to analysis model –** The design process should be traceable to the analysis model which means it should satisfy all the requirements that software requires to develop a high-quality product.
3. **Should not "Reinvent The Wheel" –** The design process should not reinvent the wheel that means it should not waste time or effort in creating things that already exist. Due to this, the overall development will get increased.
4. **Minimize Intellectual distance –** The design process should reduce the gap between real-world problems and software solutions for that problem meaning it should simply minimize intellectual distance.
5. **Exhibit uniformity and integration –** The design should display uniformity which means it should be uniform throughout the process without any change. Integration means it should mix or combine all parts of software i.e. subsystems into one system.
6. **Accommodate change –** The software should be designed in such a way that it accommodates the change implying that the software should adjust to the change that is required to be done as per the user's need.
7. **Degrade gently –** The software should be designed in such a way that it degrades gracefully which means it should work properly even if an error occurs during the execution.
8. **Assessed or quality –** The design should be assessed or evaluated for the quality meaning that during the evaluation, the quality of the design needs to be checked and focused on.
9. **Review to discover errors –** The design should be reviewed which means that the overall evaluation should be done to check if there is any error present or if it can be minimized.
10. **Design is not coding and coding is not design –** Design means describing the logic of the program to solve any problem and coding is a type of language that is used for the implementation of a design.

Master Software Testing and Automation in an efficient and time-bound manner by mentors with real-time industry experience. Join our Software Automation Course and embark on an exciting journey, mastering the skill set with ease!

Software Design strategies

Software design principles are concerned with providing means to handle the complexity of the design process effectively. Effectively managing the complexity will not only reduce the effort needed for design but can also reduce the scope of introducing errors during design.

Following are the principles of Software Design

1. Problem Partitioning

For small problem, we can handle the entire problem at once but for the significant problem, divide the problems and conquer the problem it means to divide the problem into smaller pieces so that each piece can be captured separately.

For software design, the goal is to divide the problem into manageable pieces.

Benefits of Problem Partitioning

1. Software is easy to understand
2. Software becomes simple
3. Software is easy to test
4. Software is easy to modify
5. Software is easy to maintain
6. Software is easy to expand

These pieces cannot be entirely independent of each other as they together form the system. They have to cooperate and communicate to solve the problem. This communication adds complexity.

2. Abstraction

An abstraction is a tool that enables a designer to consider a component at an abstract level without bothering about the internal details of the implementation. Abstraction can be used for existing element as well as the component being designed.

Here, there are two common abstraction mechanisms

1. Functional Abstraction
2. Data Abstraction

Functional Abstraction

i. A module is specified by the method it performs.

ii. The details of the algorithm to accomplish the functions are not visible to the user of the function.

Functional abstraction forms the basis for **Function oriented design approaches**.

Data Abstraction

Details of the data elements are not visible to the users of data. Data Abstraction forms the basis for **Object Oriented design approaches**.

3. **Modularity**

Modularity specifies to the division of software into separate modules which are differently named and addressed and are integrated later on in to obtain the completely functional software. It is the only property that allows a program to be intellectually manageable. Single large programs are difficult to understand and read due to a large number of reference variables, control paths, global variables, etc.

The desirable properties of a modular system are:

- Each module is a well-defined system that can be used with other applications.
- Each module has single specified objectives.
- Modules can be separately compiled and saved in the library.
- Modules should be easier to use than to build.
- Modules are simpler from outside than inside.

Advantages and Disadvantages of Modularity

In this topic, we will discuss various advantage and disadvantage of Modularity.

Advantages of Modularity

There are several advantages of Modularity

- It allows large programs to be written by several or different people
- It encourages the creation of commonly used routines to be placed in the library and used by other programs.
- It simplifies the overlay procedure of loading a large program into main storage.
- It provides more checkpoints to measure progress.
- It provides a framework for complete testing, more accessible to test
- It produced the well designed and more readable program.

Disadvantages of Modularity

There are several disadvantages of Modularity

- Execution time maybe, but not certainly, longer

- Storage size perhaps, but is not certainly, increased
- Compilation and loading time may be longer
- Inter-module communication problems may be increased
- More linkage required, run-time may be longer, more source lines must be written, and more documentation has to be done

Functional Independence: Functional independence is achieved by developing functions that perform only one kind of task and do not excessively interact with other modules. Independence is important because it makes implementation more accessible and faster. The independent modules are easier to maintain, test, and reduce error propagation and can be reused in other programs as well. Thus, functional independence is a good design feature which ensures software quality.

It is measured using two criteria:

- **Cohesion:** It measures the relative function strength of a module.
- **Coupling:** It measures the relative interdependence among modules.

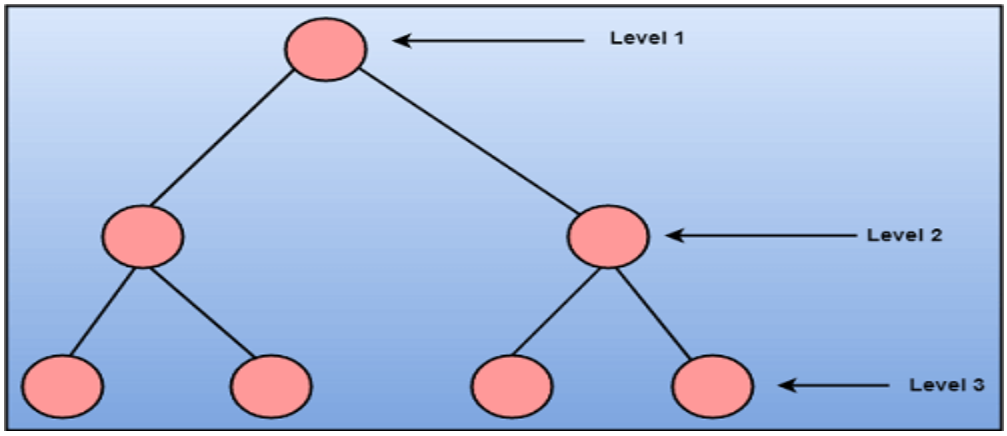
4. Top down/bottom up

A good system design strategy is to organize the program modules in such a method that are easy to develop and latter too, change. Structured design methods help developers to deal with the size and complexity of programs. Analysts generate instructions for the developers about how code should be composed and how pieces of code should fit together to form a program.

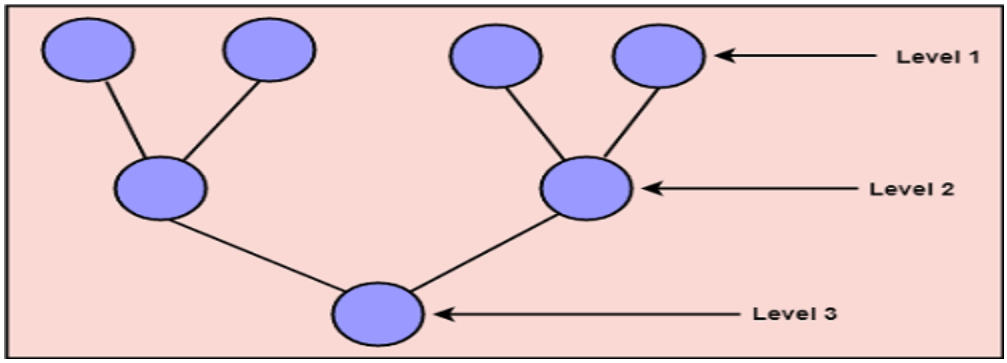
To design a system, there are two possible approaches:

1. Top-down Approach
2. Bottom-up Approach

1. Top-down Approach: This approach starts with the identification of the main components and then decomposing them into their more detailed sub-components.

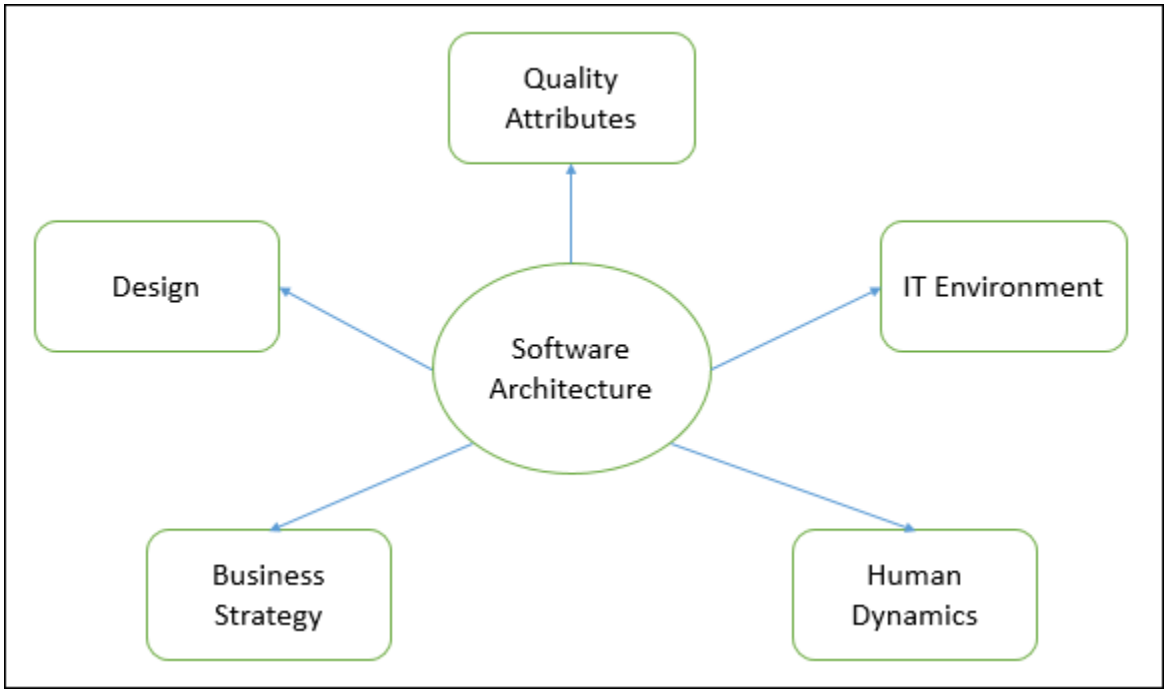


2. Bottom-up Approach: A bottom-up approach begins with the lower details and moves towards up the hierarchy, as shown in fig. This approach is suitable in case of an existing system.



Major Components: Software Architectures

The architecture of a system describes its major **components,** their relationships (structures), and how they interact with each other. Software architecture and design includes several contributory factors such as Business strategy, quality attributes, human dynamics, design, and IT environment.



Note : Refer Class lectures for All Open Ended Discussions